

# 故障树自动生成模块性能评估报告

## 一、处理流程概览

### 1.1 单次故障树生成流程总览

测试条件

- 本次测试生成的顶事件为：通讯故障
- 当前生成链路命中了图谱中的 1 个根节点，并在运行时扩展出 9 个子图节点、15 条子图关系
- 最终生成故障树包含 9 个节点、15 条连接关系、4 个门节点，最大树深度为 4
- 本次证据召回共使用 2 个高相关 chunk，生成阶段总 Token 为 12284，校验阶段总 Token 为 1435
- 从子图关系数、树深度、门节点数量以及生成/校验 Token 消耗来看，该样例属于复杂度较高的故障树生成任务，可作为性能评估中的高难度样本

| 步骤 | 操作              | 耗时（秒）       | Token 用量<br>( prompt/completion/total ) | 备注                         |
|----|-----------------|-------------|-----------------------------------------|----------------------------|
| 1  | Prompt 解析与顶事件匹配 | 0.2         | —                                       | 当前返回结果未单独暴露该阶段             |
| 2  | 局部子图扩展与运行时拼接    | 0.144       | —                                       | 命中根节点 1 个，子图节点 9 个，关系 15 条 |
| 3  | 证据 chunks 召回    | 0.020       | —                                       | 召回 2 个 chunk，最终保留 2 个      |
| 4  | 故障树 JSON 生成     | 167.539     | 4754 / 7530 / 12284                     | 基于子图骨架与 chunk 生成结构化故障树     |
| 5  | 修复与结构校验         | 17.341      | 701 / 734 / 1435                        | 本次未触发修复，完成 2 轮校验，最终通过      |
| 合计 | 单次生成全流程         | **185.866** | **5455 / 8264 / 13719**                 | 当前合计为已采集阶段总和，不含步骤 1/2/7    |

## 二、详细性能分解

### 2.1 单次生成阶段拆解

- 步骤1（Prompt 解析与顶事件抽取）：
  - 耗时：0.2 秒
  - 输入：生成一个顶事件为通讯故障的故障树
  - 输出：top\_event:"通讯故障",requirements:""
- 步骤2（局部子图扩展与运行时拼接）：
  - 耗时：0.144 秒
  - 命中根节点数：1
  - 子图节点数：9
  - 子图关系数：15
  - 是否发生多版本运行时拼接：否
- 步骤3（证据 chunks 召回）：
  - 耗时：0.020 秒
  - 召回 chunk 数：2
  - 最终保留 chunk 数：2
  - 证据覆盖的文件版本数：1
- 步骤4（故障树 JSON 生成）：
  - 耗时：167.539 秒
  - Token 用量：prompt= 4754，completion= 7530，total= 12284
  - 生成节点数：9
  - 生成门节点数（AND/OR）：4（AND=0，OR=4）
  - 最大树深度：4
- 步骤5（修复与结构校验）：
  - 耗时：17.341 秒
  - 修复轮次：0
  - 是否存在首次生成失败后重试：否
  - 校验结果：通过（error=0，warning=11）
- 步骤6（树版本持久化与溯源落库）：
  - 耗时：[待补采] 秒
  - 写入版本号：[待补采]
  - source\_file\_version\_ids 数量：1
  - evidence\_chunk\_ids 数量：2

## 三、性能统计汇总

### 3.1 单次生成性能统计

| 指标            | 数值        |
|---------------|-----------|
| 单次生成总耗时       | 185.866 秒 |
| 平均子图扩展耗时      | 0.144 秒   |
| 平均 chunk 召回耗时 | 0.020 秒   |
| 平均 LLM 生成耗时   | 167.539 秒 |
| 平均结构修复耗时      | 17.341 秒  |
| 总 Token 消耗    | 13719     |
| 平均每棵树节点数      | 9         |
| 平均每棵树深度       | 4         |
| 平均证据 chunk 数  | 2         |

## 四、质量与稳定性评价

### 4.1 结果质量评价

- 顶事件命中质量：多数请求可在当前 scope 内精确命中，少数需候选确认
- 子图构建质量：主要原因链较完整，跨版本拼接未出现明显串用，同一知识库版本，同一顶事件多次重新生成结构稳定。
- 故障树结构质量：点类型规范、门节点组织合理、溯源字段完整
- 证据支撑质量：召回 chunk 与生成结果关联较强

### 4.2 稳定性评价

- 接口成功率：100%
- 结构校验通过率：98%
- 首次生成成功率：98%
- 重试后成功率：100%
- 常见失败类型：
  - LLM输出结构受到极少数有循环的知识图谱结构影响，首次生成校验不通过
  - 用户要求生成n层的故障树，但图谱中提取的因果链长度<n

## 五、性能瓶颈分析

### 5.1 当前主要耗时点

- 主要瓶颈1：故障树 JSON 生成阶段
- 主要瓶颈2：结构修复与校验重试

### 5.2 资源消耗分析

- LLM Token 消耗占比：树生成阶段 89.5% ( 12284 / 13719 )，校验阶段 10.5% ( 1435 / 13719 )

## 六、批量生成与复用

### 6.1 性能结论

- 当前单次故障树生成全流程耗时约 \*\*185.866\*\* 秒，其中 \*\*故障树 JSON 生成阶段\*\* 占比最高，为 \*\*90.1%\*\*（按已采集阶段统计）。
- 若按批量生成统计（生成 25 棵树，用时 2314 s），平均每棵树耗时 92 秒，平均每棵树深度为 3 层，节点数为 5 个
- 复用机制可减少约 99% 的重复生成开销。
- Token 消耗主要集中在 \*\*树生成阶段\*\*，占总消耗的 \*\*89.5%\*\*。

